

stack 정리

마지막에 넣은 값이 먼저 나오는 LIFO 구조입니다.

사용처 (Where)

- 괄호 검사
- 되돌리기 기능
- DFS와 비슷한 후입선출 처리
- 최근 작업부터 처리해야 할 때

방법 (How to Use)

- push로 넣고 pop으로 제거
- top으로 맨 위 확인
- empty로 비었는지 확인 후 접근
- 내부 순회는 직접 지원하지 않음

기본 정보

- 필요 헤더: `#include <stack>`
- 기본 선언: `stack<int> st;`

왜 써야 할까? (Why)

- 후입선출 의도가 코드에 명확히 드러남
- 직접 배열로 top 인덱스를 관리하지 않아도 됨
- 괄호/뒤로가기/최근 항목 처리에 적합

주요 함수

함수/문법	의미
<code>push(x)</code>	맨 위에 추가
<code>pop()</code>	맨 위 제거
<code>top()</code>	맨 위 값 확인
<code>empty()</code>	비었는지 확인
<code>size()</code>	개수

기본 코드 예시

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<int> st;
    st.push(10);
    st.push(20);

    cout << st.top() << '\n';
    st.pop();
    cout << st.top() << '\n';
}
```

stack 비교와 응용

STL을 직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
push/pop/top	O(1)
검색	직접 지원 안 함
전체 출력	복사해서 pop하며 확인

STL vs 직접 구현 vs C 스타일

구분	STL	직접 구현	C 스타일
개발 난이도	이미 구현된 기능 사용	자료구조 설계와 예외 처리 필요	배열/포인터 직접 관리
코드 길이	짧고 읽기 쉬움	길고 버그 가능성 증가	간단하지만 확장성이 낮음
성능	일반 상황에서 충분히 최적화	구현 실력에 따라 달라짐	고정 크기에서는 단순함
안정성	범위/메모리 관리 부담 감소	메모리/인덱스 실수 가능	범위 초과 위험 큼
추천 상황	대부분의 C++ 코드	자료구조 원리 학습	아주 단순한 고정 크기 데이터

응용: 괄호 문자열 검사

여는 괄호는 push, 닫는 괄호는 top과 맞는지 확인합니다.

```
#include <iostream>
#include <stack>
using namespace std;

bool ok(string s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(') st.push(c);
        else if (c == ')') {
            if (st.empty()) return false;
            st.pop();
        }
    }
    return st.empty();
}

int main() {
    cout << ok("()");
}
```

처음 배울 때 자주 하는 실수

- empty 확인 없이 top/pop 호출 금지
- 중간 원소 접근이 필요하면 vector 사용
- 전체 순회가 목적이면 stack보다 vector가 편함

비슷한 항목과 차이

- queue는 먼저 넣은 값이 먼저 나옴
- vector로도 stack처럼 쓸 수 있지만 의도가 덜 명확
- deque가 기본 내부 컨테이너로 사용됨

한 줄 정리: stack은(는) '괄호 검사' 상황에서 먼저 떠올리면 좋은 STL입니다.